What Memory Do GUI Agents Really Need? From Passive Records to Active Task-Driving States

Chen Liu[2*], Ling Chen[2], Hanzhang Zhou[1†], Xu Zhang[1], Quyu Kong[1], Panrong Tong[1], Wenhao Wang[1], Xin Yu[3], Steven Hoi[1], Yue Wang[1†]

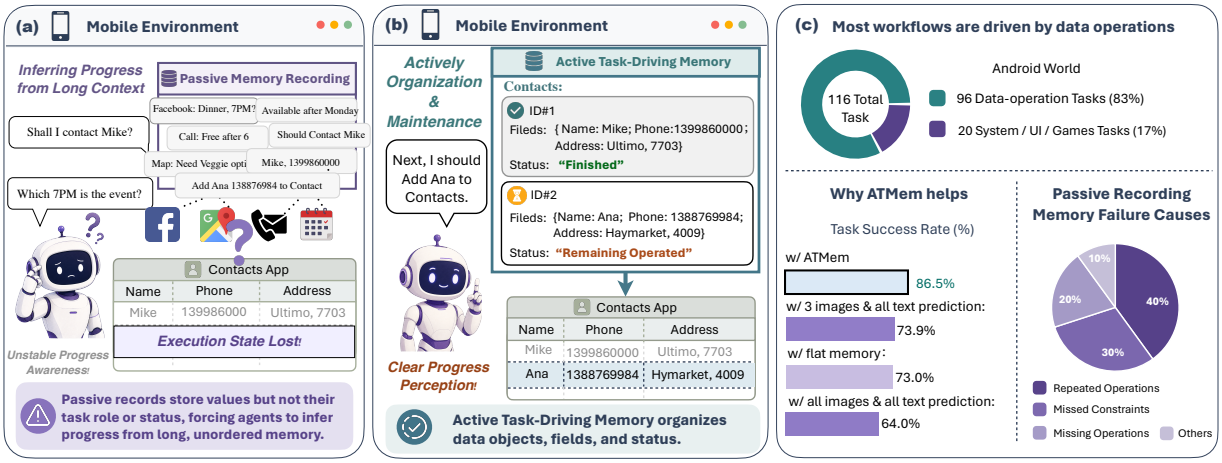

Figure 1: **Motivation.** Passive records preserve past snippets but do not provide stable execution-state awareness, leading to missed, repeated, or over-scoped operations. (c) AndroidWorld statistics show that 83% of tasks involve data operations. (d) With the same GPT-5 planner and UIIns grounding framework, ATMem improves over flat-memory and full-history baselines, and failure analysis attributes flat-memory errors mainly to repeated operations, missed operations, and forgotten constraints.

Abstract

Mobile GUI agents increasingly face long-horizon tasks that require reading, updating, and reusing task-relevant data across pages and applications. Existing methods treat memory largely as passive storage, where past observations are accumulated and retrieved when needed. Yet retrieving a value does not reveal its current role in the workflow. The agent must still infer from accumulated records whether the value should be used now, has already been used, or must wait for a later dependency. This implicit reconstruction becomes unreliable in long trajectories with repeated values, distractors, and outdated states, causing repeated or missed operations. To address this, we propose Active Task Driving Memory (ATMem), which shifts GUI-agent memory from passive storage to an actively maintained execution state. ATMem maintains task-relevant information as a continually updated execution state that links each value to its role and current status, enabling action selection based on the current workflow state. While supervised fine-tuning enables the agent to construct ATMem, it does not teach when ATMem is beneficial. We therefore introduce STR-GRPO, an online reinforcement learning method that encourages selective use of ATMem based on its contribution to task completion. STR-GRPO contrasts memory-on and memory-off rollouts to estimate when memory use improves execution, while memory-cost-aware reward discourages costly memory usage that does not improve execution. To evaluate whether agents can complete all in-scope work while avoiding out-of-scope actions, we build a challenging mobile benchmark. From a list of near identical entries, agents must act on every entry that satisfies the instruction and reject entries that violate its constraints. We further introduce *App-Level Progress* and *Scope-Aware F1* to measure these two dimensions separately. Experiments on AndroidWorld, MobileWorld, and our benchmark show that ATMem substantially improves long-horizon execution. With an 8B model, ATMem-UI achieves 76.6% success rate on AndroidWorld, outperforming UI-TARS-2-230B by 3.3 percentage points and the same-sized MAI-UI by 5.9 points. Our code and model will be publicly available.

* *Work done during an internship at Alibaba.* † *Corresponding author: yue.w@alibaba-inc.com, hanzhang.zhou@alibaba-inc.com.*
Additional affiliations: [1] *Tongyi Lab, Alibaba Group,* [2] *University of Technology Sydney,* [3] *Adelaide University.*

1. Introduction

Online mobile agents have made substantial progress in executing user instructions in real-world mobile environments, advancing from single-step grounding to multi-step navigation (Chen et al., 2024; Deng et al., 2024; Rawles et al., 2023; Sumers et al., 2023; Yang et al., 2025). As tasks extend to long-horizon workflows (Kong et al., 2025; Rawles et al., 2024), the agent must carry information across pages and applications while reading, verifying, modifying, and submitting it as the task unfolds. In such workflows, the next action often depends on information observed, generated, or modified many steps earlier that is no longer visible on the current screen (Chai et al., 2025; Chen et al., 2024; Yan et al., 2025; Yang et al., 2025).

A natural response is to equip GUI agents with memory. Existing approaches largely frame memory as a passive record of the past, whether through full interaction histories (Wang et al., 2024a,b), trajectory summaries (Xiao et al., 2026; Zhang et al., 2025), free-form notes (Wang et al., 2025b; Ye et al., 2025), or retrieval over past traces (Park et al., 2023; Shinn et al., 2023). Under this view, memory is framed as remembering more or retrieving better. We argue that this view is incomplete because making past information accessible does not necessarily make it useful for execution. As shown in Figure 1 (a), even when an agent retrieves the required value, it may still repeat a completed operation or omit a required one. Record-centric memory preserves *what was observed*, but does not bind each value to the subgoal it supports or indicate whether the value is pending, ready, or already used. The agent must therefore infer the current task state from accumulated records at every step, including which subgoals have been completed, which ones remain, and which action is currently valid. As workflows span more pages and applications, values become separated from the operations they are meant to guide, making this inference increasingly error-prone.

This raises a basic question that is rarely asked directly: ***What memory do GUI agents actually need to drive long-horizon execution?*** Inspired by human cognition, where memory is not merely a passive record of past experience but an active mechanism for maintaining goals, tracking progress, and guiding future actions (Schacter, 2012; Schacter and Addis, 2007), we argue that ***GUI-agent memory should be an active task-driving state rather than an archive of past observations.*** Guided by this insight, we propose **Active Task-Driving Memory (ATMem)**, which is updated during interaction to track task progress and guide subsequent actions. This view is particularly important in GUI workflows where task progress depends on the evolving status of task-relevant data rather than the mere availability of past observations. As shown in Figure 1 (c), 83% of AndroidWorld tasks involve data operations that advance the workflow by finding, verifying, modifying, or submitting task-relevant information. In such workflows, each completed operation changes the execution state and determines what should be done next. For example, verifying that a contact satisfies a constraint can make it eligible for the next operation, while recording one recipe from a webpage can make the next recipe the current target for transfer to another app. Similarly, once a value has been submitted, the agent should mark it as used and avoid repeating the same operation. These evolving data states determine which subgoals have been completed, which entries remain actionable, which constraints still apply, and which action is valid next. Memory for GUI agents should therefore maintain not only the values observed during execution, but also their provenance, current status, associated constraints, and role in task progress, so that memory can directly drive action selection throughout long-horizon execution.

ATMem operationalizes this idea by organizing task-relevant values, their structure, provenance, and current status into an active execution state that tracks completed subgoals, pending operations, and data available for the next action. Unlike a scratchpad or a fixed-state tracker, ATMem is not a chronological note or a hand-engineered schema. It is induced from the task and updated by the agent during interaction. ATMem maintains a hierarchical and extensible task state across app-level files and their underlying data units. **Workflow Progress** records the current phase and which app-level data files remain to be processed or have been completed. **Constraints** encode the instruction-defined logic and conditions independently of the schema. **Schema** defines

the minimal task-relevant data units, such as fields, together with their properties. **ItemContent** stores the observed content and the execution status of individual data instances. Together, these sections identify remaining files, constraint-satisfying instances, and values available for the next operation. After each new observation or operation outcome, the agent updates ATMem and combines it with the current GUI observation and interaction history to select the next action. This closed loop allows ATMem to externalize the agent's evolving execution state, helping it perceive workflow progress and maintain consistency across screens.

To equip the agent with ATMem, we train it in two stages. Supervised fine-tuning on verified trajectories teaches the agent to construct a valid ATMem structure, update it from new observations, and reference it when predicting actions. Given that this supervision specifies how ATMem should be formed but not whether relying on it improves task completion, we introduce **STR-GRPO**, a memory-aware online reinforcement learning method. During interaction, STR-GRPO learns an ATMem usage policy from task outcome rewards. It contrasts paired rollouts of the same task with ATMem enabled and disabled to estimate ATMem's contribution to task completion, while a memory cost-aware reward penalizes use that adds extra steps without improving the outcome. The agent therefore learns both how to maintain ATMem and when to rely on it for task completion.

To evaluate whether and how well agents can maintain progress and task scope over evolving task states, we build an online mobile benchmark for multi-entry tasks with scope constraints. Each task requires the agent to apply the instructed operation to every entry that satisfies the constraints while excluding entries with the same field structure that violate them. The benchmark holds the core operation schema fixed while varying the number of target entries and same-schema distractors. Difficulty therefore increases through coverage and scope control without introducing new operation types. It contains 32 manually designed templates. Using these templates, we construct three benchmark sets of increasing difficulty, each containing instances of all templates. Across these sets, the distractor-to-target ratio increases from 1.39$\times$ to 3.22$\times$. Considering that the terminal success rate is binary and hides partial progress, we introduce two complementary metrics. *App-Level Progress* measures how far the agent advances through the required operations in each application. *Scope-Aware F1* measures precision and recall over task-relevant entries while penalizing operations on irrelevant ones. Across standard benchmarks and our benchmark, ATMem and STR-GRPO yield the largest gains on workflows where progress is tightly coupled with evolving data states.

Our contributions are summarized as follows.

- We propose **Active Task-Driving Memory (ATMem)**, which represents task-relevant data as a memory actively driving execution rather than an archive of past observations, enabling explicit tracking of data ownership, task role, task constraints, execution status, and workflow progress.
- We introduce **STR-GRPO**, a memory-aware online reinforcement learning method that turns ATMem usage into a learnable policy decision through memory-on and memory-off rollout interventions and memory-cost-aware optimization.
- We build a scalable data-centric online mobile-agent benchmark that evaluates cross-page, cross-application, and data-dependent workflows. It requires agents to complete all instruction-relevant operations while filtering same-schema distractors, and introduces *App-Level Progress* to measure per-app operation completion and *Scope-Aware F1* to evaluate target coverage under distractor filtering.
- We demonstrate state-of-the-art performance with our proposed agent **ATMem-UI**. It reaches 76.6% success rate on AndroidWorld with an 8B model, outperforms UI-TARS-2-230B by 3.3 percentage points, and achieves a 36.2% relative improvement over the strongest baseline on MobileWorld.

2. Related Work

Mobile GUI Agents. Mobile GUI agents have made rapid progress in recent years (Gu et al., 2025; Qin et al., 2025; Team, 2025; Wang et al., 2025a; Xu et al., 2026; Zhou et al., 2025). AutoDroid (Wen et al., 2024)

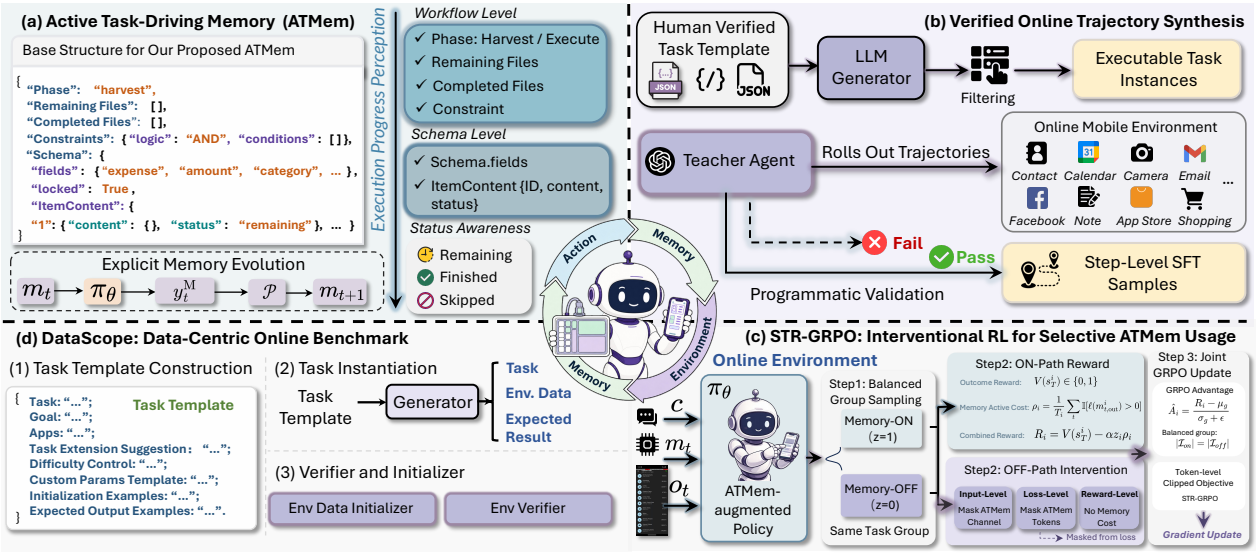

Figure 2: **Overview of our methodology.** (a) ATMem organizes task-relevant data into a structured execution state with constraints, schema fields, item content, and item-level status. (b) Verified SFT data are synthesized through task-template instantiation, teacher-agent rollouts, and environment validation. (c) STR-GRPO uses balanced memory-ON/OFF interventions to estimate ATMem utility and learn selective memory invocation. (d) DataScope evaluates data-centric workflows built from controlled target entries, same-schema distractors, and calibrated task verifiers.

formulates Android task automation as a multimodal interaction problem, and subsequent systems such as AppAgent (Zhang et al., 2025), AppAgent-v2 (Li et al., 2024), MAI-UI (Zhou et al., 2025), and Mobile-Agent-v3/v3.5 (Xu et al., 2026; Ye et al., 2025) extend agent capabilities to more complex smartphone and GUI tasks. In parallel, AndroidWorld (Rawles et al., 2024), AndroidLab (Xu et al., 2025b), and MobileWorld (Kong et al., 2025) provide realistic online environments with programmatic evaluation. More recent works, such as MobileGUI-RL (Shi et al., 2025) and MobileRL (Xu et al., 2025a), explore online reinforcement learning to improve long-horizon execution, while AgentProg (Tian et al., 2025) reduces reliance on raw history replay through programmatic context management and belief-state tracking. Together, these efforts have substantially advanced GUI interaction capabilities, online benchmarks, and long-horizon mobile-agent research.

Memory for GUI and Mobile Agents. Memory in mobile and GUI agents has developed along both experience-oriented and execution-oriented directions (Chen et al., 2026; Cheng et al., 2025; Lee et al., 2024; Li et al., 2025, 2024; Liu et al., 2026; Shi et al., 2026; Sun et al., 2025, 2026; Wang et al., 2025b; Wen et al., 2024; Xiao et al., 2026; Ye et al., 2025; Yu et al., 2026; Zhu et al., 2026, 2025). Experience-oriented methods store app knowledge (Li et al., 2024), human-like app memory (Lee et al., 2024; Wen et al., 2024), past trajectories (Zhu et al., 2025), skills (Sun et al., 2025; Wang et al., 2025b), or interaction patterns (Yu et al., 2026) to support knowledge reuse, generalization, and interface adaptation. Execution-oriented methods, such as note-taking (Wang et al., 2025b), anchored intermediate states (Zhu et al., 2026), and program-based context management (Xiao et al., 2026), preserve key information or compact task context (Shi et al., 2026; Sun et al., 2026) from ongoing trajectories to support subsequent actions. While these works highlight the importance of memory for long-horizon GUI execution, they mainly represent memory as reusable experience, recorded notes, retrieved context, or program/interface state. In contrast, our work explicitly models task-relevant data objects, their structures, values, and execution states, and uses them as actionable signals to guide execution.

3. Methods

As shown in Figure 2, our framework consists of four components. We first introduce ATMem, which turns task-relevant data from passive interaction records into structured execution states that agents actively maintain

and use to guide actions across interaction steps (Section 3.1). We then describe verified online trajectory synthesis to build high-quality SFT data (Section 3.2). Next, we present STR-GRPO for improving memory-conditioned execution (Section 3.3). Finally, we detail the construction of our DataScope benchmark for evaluating long-horizon tasks (Section 3.4).

3.1. ATMem: Active Task-Driving Memory

From Passive History to Active Task-Driving Memory. A standard mobile GUI agent can be formulated as a history-conditioned action policy. At step t, given the task instruction c, the current GUI observation o_t, and the interaction history $h_{<t}$, the agent generates a textual response y_t, from which an output parser $\mathcal{P}$ extracts an executable action a_t and an intermediate reasoning trace ρ_t:

$$y_t \sim \pi_\theta(\cdot \mid c, o_t, h_{<t}), \qquad (\rho_t, a_t) = \mathcal{P}(y_t). \tag{1}$$

Although $h_{<t}$ preserves values observed or produced in previous steps, it stores them as chronological records rather than as a task state organized around the data being processed. The agent must therefore repeatedly infer which item a value refers to, whether it satisfies the task constraints, and whether the associated work has already been completed. As trajectories grow longer and contain similar fields, repeated values, distractors, or outdated states, inferring task progress from accumulated records becomes increasingly unreliable, often leading to repeated or missed operations. This motivates a shift from passive interaction history to memory actively driving execution, where task progress, data eligibility, and remaining operations are explicitly maintained.

Our key insight is that many long-horizon GUI workflows require explicit tracking of the execution status of task-relevant data. We therefore introduce ATMem, an active task-driving memory that organizes task-relevant information as a hierarchical execution state. ATMem operationalizes this shift through a two-level structure that turns memory from a chronological record into an agent-maintained execution state. Formally, ATMem maintains a memory state $m_t = (\phi_t, E_t)$, where ϕ_t summarizes workflow-level progress and task constraints, and $E_t = \{e_i\}$ is a dynamic set of task-relevant item entries. Each entry e_i stores a stable item identifier, structured task-critical content, and an execution status $s_i \in \{\texttt{remaining}, \texttt{finished}, \texttt{skipped}\}$, indicating whether the item still needs to be processed, has been completed, or should be skipped. ATMem therefore represents not only the available data, but also the item-level state needed to determine remaining work.

Before each model call, the current ATMem state m_t is inserted into the system prompt as the current task-driving memory. Conditioned on this injected memory, the agent generates a memory-augmented response y_t^{M}. The parser $\mathcal{P}$ then extracts the updated task-driving memory m_{t+1}, the intermediate reasoning trace ρ_t, and the executable GUI action a_t from the model response:

$$y_t^{\mathrm{M}} \sim \pi_\theta(\cdot \mid c, o_t, h_{<t}, m_t), \qquad (m_{t+1}, \rho_t, a_t) = \mathcal{P}(y_t^{\mathrm{M}}). \tag{2}$$

The extracted m_{t+1} is then inserted into the next system prompt as the current memory, enabling the agent to carry forward an up-to-date execution record without relying on history re-examination.

Basic Structure Design. As illustrated in Figure 2 (a), ATMem organizes memory into two complementary levels that implement the shift from passive history to active task-driving state. Workflow-level fields encode where the agent is in the task and what global constraints should guide execution, while schema-level fields define and track the minimal actionable data units on which GUI operations are performed. A minimal actionable data unit is the smallest semantically complete data object that can be independently operated on during task execution. For example, an expense record comprising `expense`, `amount`, and `category` forms one such unit, whereas any individual attribute alone is insufficient to constitute an operable target.

① **Workflow-level fields.** Workflow-level fields capture task-level execution progress and global task conditions with a fixed structure shared across tasks. The `Phase` field partitions execution into two macro-stages, `harvest` and `execute`. During `harvest`, the agent inspects available sources and populates task-relevant item entries.

During `execute`, the agent performs data-dependent operations over the populated entries. The phase transitions from `harvest` to `execute` once sufficient items have been collected and no sources remain to be inspected. For tasks that require source-level tracking, `RemainingFiles` and `CompletedFiles` maintain a checklist of sources pending inspection and sources already processed, providing the concrete signal for phase transition. When source-level tracking is not required, both fields remain empty. The `Constraints` field stores task-derived filtering conditions as global execution rules, determining which collected items are eligible for operation, such as selecting only records that satisfy a specified date range, category, or recipient.

② **Schema-level fields.** Schema-level fields define the minimal actionable data units for a given task and maintain their execution states. `Schema.fields` specifies the attribute set that constitutes one semantically complete data unit, such as `expense`, `amount`, `category` in an expense-management task or `recipient`, `subject` in an email task. During schema construction, explicit task-specific attributes can be added when required. Once the relevant attributes are identified, `Schema.locked` freezes the field set to prevent structural drift across steps. `Schema.ItemContent` enumerates instantiated data units, each indexed by a stable identifier and associated with structured field values and an execution status from {`remaining`, `finished`, `skipped`}. Here, `remaining` and `finished` indicate whether a unit still awaits processing or has been completed. `skipped` provides an item-level refinement complementary to `Constraints`. While `Constraints` captures global eligibility rules derived from the task instruction, `skipped` allows the agent to mark a specific item as inapplicable when richer item-level details become available during execution.

Together, workflow-level fields control task flow and global filtering conditions, while schema-level fields define and track the concrete data units to be operated on. The base ATMem structure provides a shared template across tasks, while task-specific fields can be added to accommodate different workflows, such as date attributes for calendar tasks. We provide the execution prompt in Appendix A.3.

3.2. Verified Online Rollout Synthesis.

As shown in Figure 2 (b), we synthesize supervised data through verifier-gated online rollouts in a virtual mobile environment with 20 applications. We construct 120 task templates, each human-designed and verified, to define a controllable and verifiable task space rather than treating tasks as isolated natural-language instructions. Each template consists of two executable components and a set of instantiation guidelines. The initializer constructs the required environment state by injecting task-relevant data and distractors, and the task-specific verifier determines whether the terminal environment state satisfies the intended goal. The guidelines include a goal pattern, descriptions of required data objects and fields, task-instance examples, and suggested expansion directions to support downstream instantiation.

Based on these templates, an LLM-assisted generator instantiates about 1.2K candidate task instances with user-facing instructions and concrete initialization data. The generator leverages template-provided examples and expansion directions to produce valid task variations, while executability and correctness are determined by the initializer and verifier. After removing unreachable, malformed, or invalid instances, we obtain 1.1K executable task instances for rollout collection.

For each executable task instance, we run an online rollout framework that combines an LLM planner (OpenAI, 2025) with UIIns (Chen et al., 2025), a UI grounding model that maps predicted operations to executable GUI actions. The planner is instructed to maintain ATMem only when structured memory is needed, such as in workflows involving multiple data items, cross-file information collection, or repeated data-dependent operations. When ATMem is not needed, the memory state is kept empty. At each step, the rollout framework produces a response containing the updated memory state, an intermediate reasoning trace, and an executable GUI action. A trajectory is retained only if its terminal environment state passes the corresponding task verifier. This verifier-gated process yields 21,713 step-level SFT samples. Each sample maps the current execution context $(c, o_t, h_{<t}, m_t)$ to the target response $y_t = (m_{t+1}, \rho_t, a_t)$, where m_t and m_{t+1} are empty when ATMem is not activated.

3.3. Interventional Online RL for Effective ATMem Usage

Online RL Setup. After SFT on verifier-accepted trajectories, the agent can construct and update ATMem during GUI execution. However, SFT only imitates memory patterns observed in successful rollouts and does not estimate whether ATMem contributes to the current policy's execution. We therefore introduce Structured Task-driven Reward GRPO (STR-GRPO), an interventional online RL method. During paired rollouts, we intervene on the explicit ATMem channel by enabling or masking memory access while keeping the task initialization unchanged. The resulting rewards provide a relative estimate of ATMem's execution utility.

Memory-Conditional Intervention. Given a task q, STR-GRPO samples a group of N_g rollouts from the current policy and assigns each rollout to one of two memory conditions. Let $z_i \in \{0, 1\}$ denote the intervention for rollout i, where $z_i = 1$ enables the explicit ATMem channel and $z_i = 0$ masks it. At step t, the policy input can be formulated as:

$$x_t^{(z_i)} = (c, o_t, h_{<t}, \tilde{m}_t^{(z_i)}), \qquad \tilde{m}_t^{(z_i)} = \begin{cases} m_t, & z_i = 1, \\ \varnothing, & z_i = 0. \end{cases} \tag{3}$$

Here, c is the task instruction, o_t is the current observation, and $h_{<t}$ denotes the standard interaction history. The intervention changes only the explicit memory channel, while previous observations, executed actions, and reasoning traces remain available in both conditions. To reduce the effect of task difficulty, we apply a balanced assignment within each group, with $N_g/2$ rollouts assigned to each condition.

Reward Design and STR-GRPO Objective. Given the balanced memory intervention above, STR-GRPO assigns each rollout a trajectory-level reward that combines terminal task validation and a memory-active cost:

$$R_i = V(s_{T_i}^i) - \alpha z_i \rho_i, \qquad \rho_i = \frac{1}{T_i} \sum_{t=1}^{T_i} \mathbb{I}\left[\ell(m_{t,\text{out}}^i) > 0\right]. \tag{4}$$

Here $V(s_{T_i}^i) \in \{0, 1\}$ is the verifier reward for the terminal environment state, $m_{t,\text{out}}^i$ denotes the ATMem block generated at step t, and $\ell(m) = 0$ if and only if all fields of the ATMem block are empty. Thus, ρ_i measures the fraction of memory-active steps, and the gate z_i restricts the memory cost to memory-ON rollouts, where ATMem is propagated across turns.

For each rollout group g, we compute the group-normalized advantage over the balanced mixture of memory-ON and memory-OFF trajectories $\hat{A}_i = (R_i - \mu_g)/(\sigma_g + \epsilon_A)$, where μ_g and σ_g are the mean and standard deviation of rewards within group g, and ϵ_A is a small constant for numerical stability. This provides a shared baseline for both memory conditions within the same task group, so that advantages directly reflect the marginal utility of the ATMem channel.

Overall, the STR-GRPO objective is defined as:

$$\mathcal{L}(\theta) = -\mathbb{E}\left[\sum_{i,t,k} \min\Big(r_{t,k}\,\hat{A}_i,\ \text{clip}(r_{t,k}, 1 \pm \varepsilon)\,\hat{A}_i\Big)\right], \quad r_{t,k} = \frac{\pi_\theta(y_{t,k}^i \mid x_t^i, y_{t,<k}^i)}{\pi_{\theta_{\text{old}}}(y_{t,k}^i \mid x_t^i, y_{t,<k}^i)}. \tag{5}$$

For memory-ON rollouts, the loss covers the ATMem block, reasoning trace, and executable action. For memory-OFF rollouts, ATMem block tokens are masked from the loss, and the loss covers only the reasoning trace and executable action under an empty memory channel. Memory-ON rollouts receive higher relative advantages only when ATMem improves verifier reward enough to offset the memory-active cost, discouraging redundant memory dependence and promoting selective structured memory usage.

3.4. DataScope Online Mobile Benchmark and Metrics

Benchmark Overview. We introduce DataScope, an online mobile benchmark for multi-entry tasks with controlled scope difficulty. Each task requires the agent to apply the instructed operations to every entry that

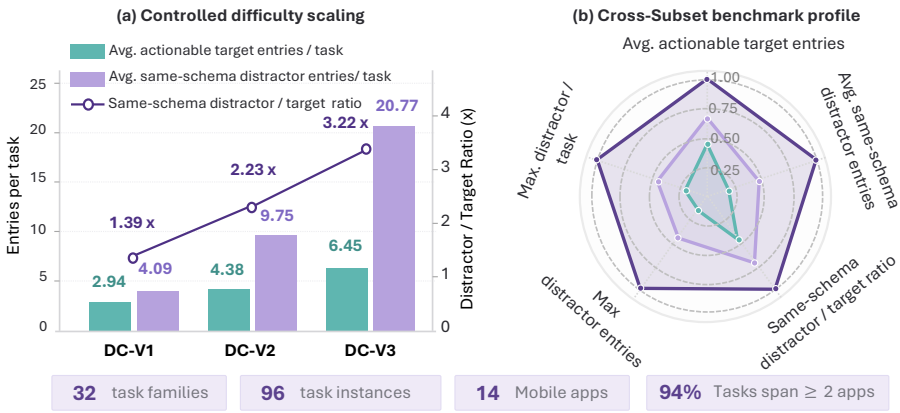

Figure 3: **DataScope statistics and controlled difficulty scaling. (a)** Bars show the average numbers of actionable target entries and same-schema distractors per task from DC-V1 to DC-V3, while the line shows the distractor-to-target ratio. The controlled increase in both quantities raises the ratio from $1.39\times$ to $3.22\times$, testing whether agents can maintain target coverage while filtering increasingly confusable data within the same task families. **(b)** Radar profiles summarize average target and distractor counts, their ratio, and the maximum distractor load and ratio across difficulty levels. The outward shift from DC-V1 to DC-V3 indicates progressively greater data-coverage and distractor-filtering demands. DataScope contains 32 task families, 96 task instances, and 14 mobile apps, with 94% of tasks spanning at least two apps.

satisfies the instruction constraints while excluding structurally matched distractors that share the same field schema or data type. Unlike prior benchmarks that evaluate end-to-end GUI task completion, DataScope controls both target coverage and distractor filtering while holding the underlying operation schema fixed.

As shown in Figure 3, DataScope contains 32 manually designed task templates, each instantiated at three difficulty levels from DC-V1 to DC-V3. The difficulty levels increase the number of target entries and structurally matched distractors without introducing new operation types. The distractor-to-target ratio increases from $1.39\times$ in DC-V1 to $3.22\times$ in DC-V3, requiring broader target coverage and more precise distractor filtering. In total, DataScope includes 96 evaluation instances across 14 apps, with 94% of tasks spanning at least two apps.

Task Construction. Each task template is defined by a human-verified workflow specification:

$$\Omega = (\mathcal{A}, g_0, \mathcal{X}, \mathcal{C}_\ell, I_\Omega, \mathcal{S}_{\text{exp}}, \mathcal{B}_{\text{val}}) \,. \tag{6}$$

Here $\mathcal{A}$ specifies the apps involved in the workflow, and g_0 defines the task objective to be completed by the agent. $\mathcal{X}$ describes the structure and seed examples of task data, including target entries that must be operated on and confusable distractors that must remain unchanged. $\mathcal{C}_\ell$ defines the data and operation constraints for difficulty level ℓ. The initializer I_Ω writes an instantiated data sample into the corresponding app storage. $\mathcal{S}_{\text{exp}}$ specifies the terminal-state requirements for successful execution, while $\mathcal{B}_{\text{val}}$ contains controlled environment states and their expected evaluation scores for verifier calibration.

Our task construction proceeds in three stages. **First**, the workflow specification Ω is manually constructed and verified. **Second**, an LLM-assisted generator instantiates executable task instances from Ω. Each instance consists of a task instruction c, task-specific initialization data x, and a terminal-state specification $y^\star$. The initializer I_Ω writes x, including target entries and confusable distractors, into the corresponding app storage to construct the initial environment state. The agent must therefore access and manipulate all task-relevant data through real GUI interaction. **Third**, we generate a functional verifier for each task template. The LLM framework synthesizes the verifier from the initialization function, terminal-state requirements, and validation basis. For calibration, $\mathcal{B}_{\text{val}}$ provides controlled environment states covering successful completion, missing required operations, over-operation on distractors, missing app-level operations, and incorrect field values. Each state is paired with its expected terminal success, App-Prog., and Scope-F1 scores. A generated

verifier is validated against all controlled states. If its outputs do not match the expected scores, the verifier is manually corrected and revalidated until it passes all calibration cases. Further construction details and template examples are provided in Appendix A.4, A.6, and A.5.

App- and Data-Level Metrics. Terminal success equals one only when all task-specific terminal-state requirements are satisfied. Although appropriate for measuring complete execution, it assigns the same zero score to unsuccessful trajectories that terminate at different stages or exhibit different data-operation errors. We therefore introduce two complementary metrics at finer levels of granularity. App-Prog measures the completion of app-level subgoals, while Scope-F1 evaluates the completeness of individual data operations.

For task q, let $\mathcal{A}_q$ be the set of apps involved, and let $c_{q,a} \in \{0, 1\}$ indicate whether all required operations in app a are completed. *App-Prog.* measures the fraction of apps with all required operations satisfied:

$$S_{\text{prog}}(q) = \frac{1}{|\mathcal{A}_q|} \sum_{a \in \mathcal{A}_q} c_{q,a}. \tag{7}$$

An app contributes only when its complete required subgoal is achieved. App-Prog therefore provides coarse-grained progress awareness across a multi-app workflow and distinguishes trajectories that complete different numbers of required applications. It focuses only on required subgoal completion, while unintended operations are evaluated separately by Scope-F1.

Scope-F1 evaluates execution at the atomic data-unit level. For each app a, let $\mathcal{D}^{\star}_{q,a}$ denote the set of required atomic data units and $\hat{\mathcal{D}}_{q,a}$ the set of data units correctly handled by the agent. Each unit is the smallest independently verifiable element of task execution, jointly identified by its data entry, field, and expected value where applicable. A unit is counted as correctly handled only when the final environment state reflects the intended change on the intended entry. We define $\text{TP}_{q,a} = \left|\hat{\mathcal{D}}_{q,a} \cap \mathcal{D}^{\star}_{q,a}\right|$, $\text{FP}_{q,a} = \left|\hat{\mathcal{D}}_{q,a} \setminus \mathcal{D}^{\star}_{q,a}\right|$, $\text{FN}_{q,a} = \left|\mathcal{D}^{\star}_{q,a} \setminus \hat{\mathcal{D}}_{q,a}\right|$, and compute the per-app F1 as $F_{q,a} = 2\,\text{TP}_{q,a} \,/\, (2\,\text{TP}_{q,a} + \text{FP}_{q,a} + \text{FN}_{q,a} + \epsilon)$, where ϵ is a small constant for numerical stability. Let $\mathcal{A}^{\text{act}}_q$ denote the set of apps containing at least one data unit modified by the agent. Scope-F1 is macro-averaged over both required and additionally modified apps:

$$S_{\text{scope}}(q) = \frac{1}{|\mathcal{A}_q \cup \mathcal{A}^{\text{act}}_q|} \sum_{a \in \mathcal{A}_q \cup \mathcal{A}^{\text{act}}_q} F_{q,a}. \tag{8}$$

Missing required units are counted as false negatives. Distractor entries, incorrect fields, incorrect values, and changes in unrelated apps are counted as false positives. Including additionally modified apps ensures that data changes outside the required workflow are also penalized.

Terminal success, App-Prog, and Scope-F1 evaluate execution at the task, application, and atomic data-unit levels, respectively. App-Prog indicates how far the agent progresses across the required applications, while Scope-F1 measures whether the required data units are correctly completed without modifying data outside the intended scope.

4. Experiments

4.1. Experimental Setup

Training Environment and Infrastructure. We use a virtual Android platform containing 20 applications for both SFT trajectory collection and online RL training. SFT trajectories are retained only after successful task-specific verification. For online RL, we build an asynchronous on-policy training arena with 128 active containerized Android virtual devices distributed across multiple bare-metal servers. Multi-turn rollouts are generated by the latest policy and asynchronously collected to reduce idle time.

Policy optimization is performed on 128 NVIDIA H20 GPUs using hybrid parallelism to support long multimodal trajectories containing GUI observations, reasoning traces, actions, and ATMem states. This infrastructure

Table 1: Performance on **AndroidWorld**. SR denotes success rate. Green/purple indicates the best/second-best results within the same scale group.

Method	Params	SR
Larger and proprietary models for reference		
UI-TARS-SFT (Qin et al., 2025)	72B	65.9
UI-Venus (Gu et al., 2025)	72B	65.9
Qwen3-VL-235B-A22B (Bai et al., 2025)	235B	63.7
UI-TARS-1.5 (Seed, 2025b)	–	64.2
Gemini-2.5-Pro (DeepMind, 2025)	–	69.7
Seed1.8 (Seed, 2025a)	–	70.7
UI-TARS-2 (Wang et al., 2025a)	230B	73.3
Recent 4B/8B models		
Step-GUI (Yan et al., 2025)	8B	67.7
MAI-UI (Zhou et al., 2025)	8B	70.7
GUI-Owl-1.5-Thinking (Xu et al., 2026)	8B	**71.6**
ATMem-UI (Ours)	4B	65.5
ATMem-UI (Ours)	8B	**76.6**

Table 2: Generalization on **MobileWorld** in unseen environments. Green/purple indicates the best/second-best results within the same scale group.

Method	Params	SR
Larger models for reference		
UI-Venus-1.5 (Team et al., 2026)	30B	17.1
UI-Venus (Gu et al., 2025)	72B	16.4
GUI-Owl (Ye et al., 2025)	72B	8.5
Recent 4B/7B/8B models		
GUI-Owl (Ye et al., 2025)	7B	7.7
UI-Venus (Gu et al., 2025)	7B	8.5
GELab-Zero (Team, 2025)	4B	16.1
ATMem-UI (Ours)	4B	**20.5**
ATMem-UI (Ours)	8B	**23.3**

enables scalable rollout collection and end-to-end optimization for long-horizon mobile interaction. Further implementation details and hyperparameters are provided in Appendix A.1 and Appendix A.2.

Evaluation Benchmarks. We evaluate our models on AndroidWorld and MobileWorld, two widely used online mobile-agent benchmarks, together with our proposed DataScope benchmark. AndroidWorld and MobileWorld evaluate general online mobile-agent performance, while DataScope focuses on cross-app workflows that require target data identification, distractor filtering, and stateful data manipulation.

DataScope contains three difficulty levels, instantiated from the same 32 task families by progressively increasing target entries, confusable distractors, and required operations. Each level contains 32 task instances, yielding 96 instances across 14 mobile apps. Overall, 94% of DataScope tasks involve at least two apps.

Baselines. Our 4B and 8B models are initialized from Qwen3-VL-4B and Qwen3-VL-8B, respectively (Bai et al., 2025). We compare them with the corresponding Qwen3-VL backbones and representative mobile agents, including GUI-Owl (Xu et al., 2026; Ye et al., 2025), UI-Venus (Gu et al., 2025), Doubao-1.5-UI-TARS (Qin et al., 2025), and GELab-Zero (Team, 2025).

We additionally report a strong teacher-agent reference that uses GPT-5 (OpenAI, 2025) for high-level planning and UIIns (Chen et al., 2025) for visual grounding. Because this system relies on a substantially stronger proprietary planner, we treat it as a teacher reference rather than a size-matched baseline.

4.2. Main Results

Performance on General Online Benchmarks. As shown in Table 1, ATMem-UI-8B achieves 76.6% SR on AndroidWorld, outperforming all recent 4B/8B agents by a clear margin, with gains of +5.0 points over GUI-Owl-1.5-Thinking and +5.9 points over MAI-UI-8B. More tellingly, it surpasses UI-TARS-2-230B by 3.3 points while using roughly $1/29$ of its parameters. This disproportionate gain suggests that the bottleneck in long-horizon mobile execution is not only model capacity.

The 4B results further illustrate the scale efficiency of structured memory. ATMem-UI-4B achieves 65.5% SR, within 0.4 points of the 72B-scale UI-TARS-SFT and UI-Venus, and exceeds Qwen3-VL-235B-A22B. This comparison suggests that explicit state tracking can compensate for part of the capability gap usually addressed by parameter scaling, especially in long-horizon tasks where failures often arise from losing intermediate task state rather than from single-step perception or action selection alone.

Table 2 evaluates cross-environment generalization on MobileWorld, where all environments are unseen during training. ATMem-UI-8B achieves 23.3% SR, outperforming the strongest non-ours baseline, GELab-Zero-4B, by 7.2 points and exceeding the 30B reference, UI-Venus-1.5, by 6.2 points. The 4B variant already surpasses all listed non-ours baselines, including 72B references. This strong transfer suggests that ATMem does not

Table 3: Performance on our benchmarks. We additionally provide a strong agentic framework as a competitive reference baseline. SR denotes success rate, S_{prog} denotes progress score, and S_{scope} denotes scope score. Green/purple indicates the best/second-best results among end-to-end mobile agents.

Method	Params	Data-Scope-V1			Data-Scope-V2			Data-Scope-V3		
		SR	S_{prog}	S_{scope}	SR	S_{prog}	S_{scope}	SR	S_{prog}	S_{scope}
Agentic framework for reference										
GPT-5.2+ UIIns (Chen et al., 2025; OpenAI, 2025)	–	18.8	35.4	38.4	6.2	16.1	21.3	3.1	9.3	13.2
End-to-end mobile agents										
GELab-Zero (Team, 2025)	4B	0.0	4.2	3.9	0.0	3.6	6.4	0.0	2.6	4.4
UI-TARS-1.5 (Seed, 2025b)	7B	0.0	1.6	1.7	0.0	1.6	1.8	0.0	1.6	1.5
GUI-Owl (Ye et al., 2025)	7B	0.0	3.6	5.7	0.0	1.6	1.1	0.0	1.6	1.5
UI-Venus (Gu et al., 2025)	7B	0.0	1.6	3.8	0.0	1.6	0.6	0.0	0.0	0.7
GUI-Owl-1.5 (Xu et al., 2026)	8B	**3.1**	4.7	7.3	0.0	2.6	**7.4**	0.0	2.6	**5.0**
MAI-UI (Zhou et al., 2025)	8B	**3.1**	**8.8**	**10.7**	**3.1**	**4.2**	3.9	0.0	**3.7**	4.2
GUI-Owl (Ye et al., 2025)	32B	0.0	2.6	2.2	0.0	1.6	1.1	0.0	0.0	0.5
UI-Venus-1.5-A3B (Team et al., 2026)	30B	0.0	1.6	4.1	0.0	1.6	1.8	0.0	1.6	1.5
Our model										
ATMem-UI (Ours)	**8B**	**6.2**	**11.7**	**15.7**	**6.2**	**9.4**	**9.8**	**3.1**	**6.3**	**8.2**

merely memorize training-app interaction patterns; instead, it provides a more portable representation of task progress and task-relevant data state, enabling better generalization to new mobile applications and workflows.

Performance on our benchmark. DataScope directly probes the capabilities ATMem is designed to support, including tracking target data items across sources and maintaining execution scope over long cross-app workflows. As shown in Table 3, existing end-to-end agents struggle substantially on all difficulty levels.

On DC-V1, the strongest end-to-end baseline MAI-UI-8B obtains only 3.1% SR, 8.8% S_{prog}, and 10.7% S_{scope}. ATMem-UI-8B improves these to 6.2%, 11.7%, and 15.7%, yielding gains of +3.1, +2.9, and +5.0 points, respectively. The larger gain on S_{scope} is especially informative: it indicates that ATMem improves not only whether the agent makes progress, but also whether it operates on the correct set of required data items. This is precisely the failure mode targeted by ATMem's explicit `Constraints` field and item-level status tracking, which provide a persistent record of which data objects remain eligible, completed, or skipped.

As difficulty increases from DC-V1 to DC-V3, the distractor-to-target ratio rises from $1.39\times$ to $3.22\times$, and all agents' performance drops sharply, confirming that denser distractors and tighter data dependencies substantially increase task difficulty. ATMem-UI-8B degrades more slowly than all end-to-end baselines, maintaining the best end-to-end results at every level. On DC-V2, it achieves 6.2% SR, 9.4% S_{prog}, and 9.8% S_{scope}, improving over the best non-ours end-to-end result for each metric by 3.1, 5.2, and 2.4 points, respectively. On DC-V3, it achieves 3.1% SR, 6.3% S_{prog}, and 8.2% S_{scope}, with corresponding gains of 3.1, 2.6, and 3.2 points. The gains on both S_{prog} and S_{scope} show that ATMem improves not only how far the agent progresses, but also how reliably it preserves the correct operation scope under dense distractors. This pattern is consistent with the role of ATMem's `Constraints` field and `skipped` status, which

Table 4: Ablation study on Qwen3-VL-8B-Instruct. SR denotes success rate. Mem. denotes the fraction of tasks in which the agent invokes ATMem at least once during execution. Green/purple highlights the best/second-best results. Green Δ values indicate SR increases, while red Δ values indicate Mem. reductions.

Training Recipe	AndroidWorld		MobileWorld	
	SR (%)	Mem. (%)	SR (%)	Mem. (%)
Qwen3-VL-8B-Instruct				
Baseline	47.6	–	9.4	–
SFT	70.7	48.2	17.2	49.6
Δ vs. Baseline	+23.1	–	+7.8	–
SFT + GRPO	**74.8**	**45.7**	**20.7**	**44.7**
Δ vs. SFT	+4.1	-2.5	+3.5	-4.9
SFT + STR-GRPO	**76.6**	**31.9**	**23.3**	**30.1**
Δ vs. SFT + GRPO	+1.8	-13.8	+2.6	-14.6
Δ vs. SFT	+5.9	-16.3	+6.1	-19.5

Figure 4: **Execution case of ATMem-UI on AndroidWorld.** The figure shows how our agent collects task-relevant data, maintains their structured execution states, and uses these states to guide subsequent actions. By tracking which data items are pending or completed, the agent reliably progresses from data collection to task execution and completes the long-horizon workflow.

help maintain item-level eligibility information throughout execution and become increasingly important as distractor density grows.

We further include GPT-5.2+UIIns as a framework-level reference built on a strong proprietary planner with explicit external orchestration. It substantially outperforms all end-to-end agents on DC-V1, achieving 18.8% SR, 35.4% S_{prog}, and 38.4% S_{scope}. However, its SR also drops from 18.8% on DC-V1 to 3.1% on DC-V3, matching ATMem-UI-8B at the highest difficulty level. This convergence in task success is revealing: under high distractor density, stronger external orchestration alone is not sufficient to avoid failure, suggesting that fine-grained data discrimination and item-level eligibility judgment become critical bottlenecks. At the same time, the framework retains higher S_{prog} and S_{scope}, indicating better partial execution quality and pointing to richer intermediate state representation as an important direction for end-to-end agents. The consistent gains of ATMem-UI across all difficulty levels demonstrate that structured memory is an effective step in this direction, without relying on external orchestration or proprietary planners.

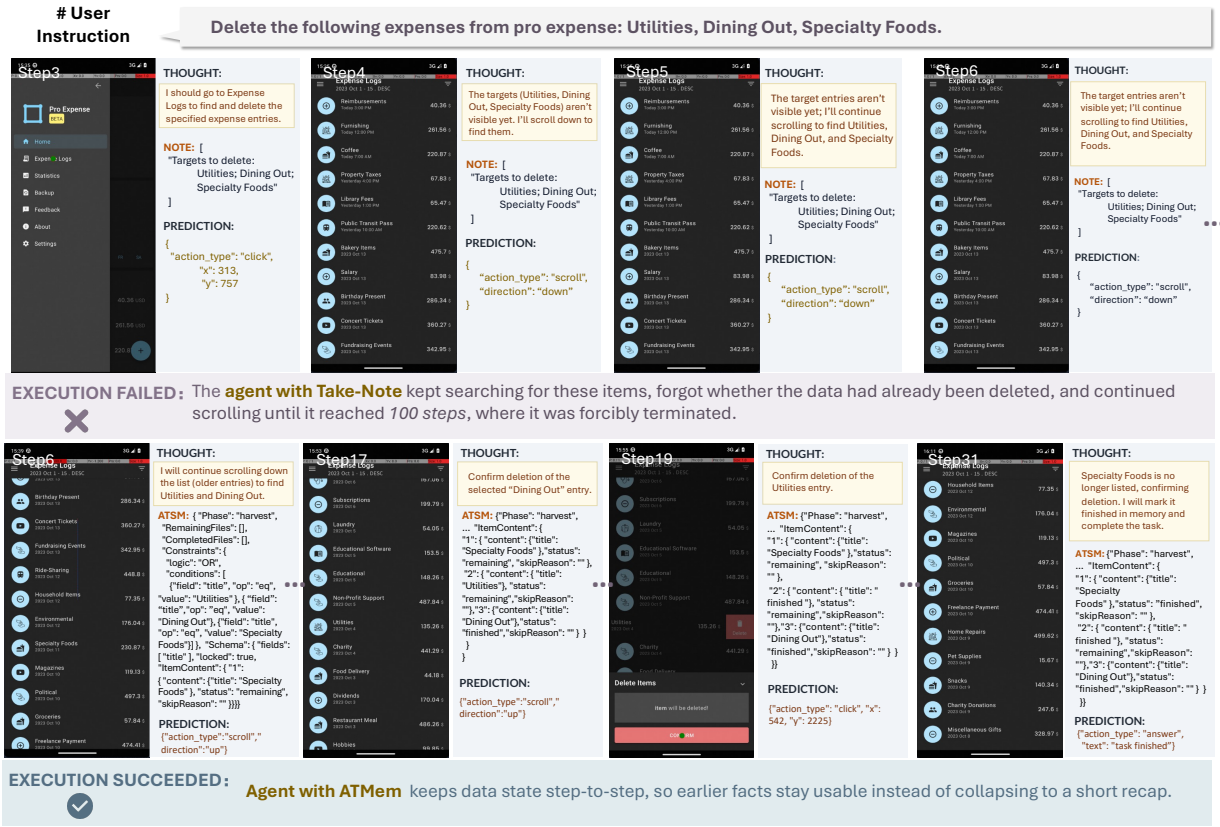

Figure 5: **Execution trajectory comparison between recording-centric memory and ATMem.** The flat-memory agent records task information as unstructured notes, but fails to explicitly track which data items have been completed or still require action, leading to repeated search and stuck execution. In contrast, the ATMem-based agent maintains structured task data and item-level execution status, enabling more stable progress across complex operations. Due to space limits, we omit part of the memory content in the figure.

4.3. Ablation Study

We conduct an ablation study on Qwen3-VL-8B-Instruct to isolate the contribution of each training stage. In addition to SR, we report Mem., defined as the fraction of tasks in which the agent invokes ATMem at least once during execution, to measure how selectively the model applies structured memory across different tasks.

As shown in Table 4, the baseline achieves 47.6% and 9.4% SR on AndroidWorld and MobileWorld, respectively. After SFT, SR increases substantially by 23.1 and 7.8 points on the two benchmarks, with Mem. reaching 48.2% and 49.6%. To encourage more selective and effective memory invocation, we introduce STR-GRPO, which estimates the marginal utility of ATMem through paired memory-ON and memory-OFF rollouts.

As Table 4 suggests, Standard GRPO further improves SR by 4.1 and 3.5 points, and modestly reduces Mem. by 2.5 and 4.9 points. The reduction is real but limited. Through online interaction, the agent receives indirect feedback that some memory-active trajectories do not perform better than simpler ones, which weakly discourages unnecessary invocations. However, because task-level reward does not distinguish whether a successful trajectory succeeded due to ATMem or despite it, the agent cannot reliably identify which memory invocations are causally useful and which are redundant. As a result, Mem. remains elevated and the agent continues to apply the ATMem structure to tasks even without extra benefit.

STR-GRPO addresses this issue by estimating the marginal utility of ATMem through paired memory-ON and memory-OFF rollouts within each task group. This design provides an explicit signal for whether structured memory improves the verifier outcome, allowing the agent to suppress memory updates that do not contribute

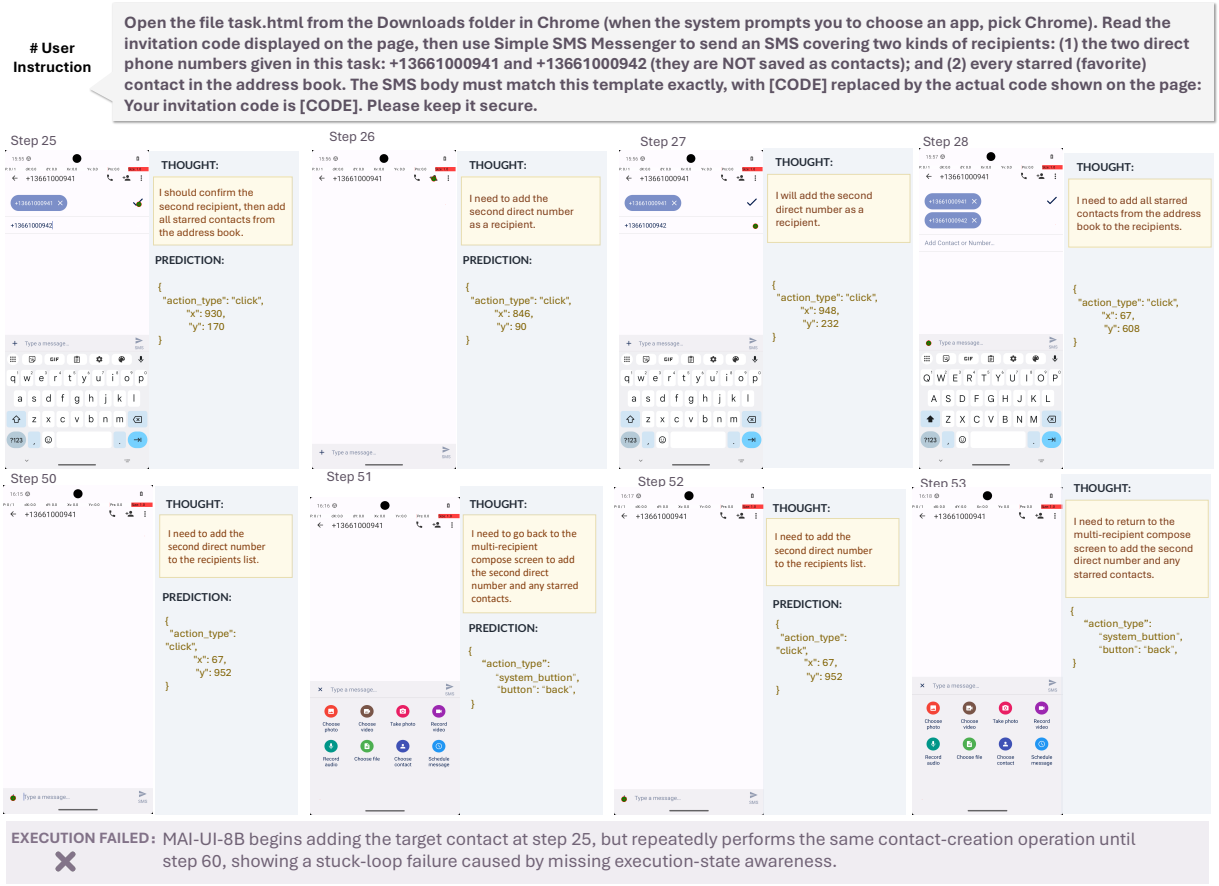

Figure 6: **Representative failure case on our data-scope benchmark.** On a data-scope workflow from our benchmark, MAI-UI-8B identifies the target contact information and begins adding the contact at step 25, but then repeats the same contact-creation operation until step 60. This stuck-loop behavior suggests that the agent can recover relevant values, but fails to track whether the data operation has already been completed.

to success. Compared with standard GRPO, STR-GRPO reduces Mem. by 13.8 and 14.6 points while further improving SR by 1.8 and 2.6 points. This joint improvement indicates that many SFT-induced ATMem updates are redundant, and that STR-GRPO removes unnecessary invocations while preserving memory use where it is beneficial. Relative to SFT, STR-GRPO reduces Mem. by 16.3 and 19.5 points while improving SR by 5.9 and 6.1 points. These results show that STR-GRPO does not simply discourage memory use, but recalibrates ATMem from a frequently imitated behavior into a more selective, utility-driven mechanism.

4.4. Qualitative Analysis

Flat Notes vs. ATMem. To isolate the effect of memory structure, we run the same agentic framework on identical tasks, changing only the memory format. One setting uses recording-centric memory(Wang et al., 2025b), while the other uses ATMem. Figure 5 shows a representative task where the agent must delete three specific expense entries from a list containing many same-schema distractors. The recording-centric agent stores the target names as a static list and starts searching for them. However, this memory format does not track which entries have already been deleted. After removing some targets, the agent revisits previously processed regions and eventually reaches the 100-step limit. This is a state-tracking failure. Without item-level completion status, the agent cannot reliably determine which targets remain.

ATMem changes the execution pattern by maintaining explicit status for each target item. The agent instantiates the three expense entries under `Schema.ItemContent`, assigns each `status:remaining`, and encodes the target condition in the `Constraints` field. After each deletion, the corresponding item is updated to `status:finished`,

so later decisions are conditioned on the remaining unfinished items. The task is completed in 31 steps. This comparison suggests that structured status tracking converts open-ended search into a bounded checklist, reducing the ambiguity that causes flat-note agents to loop.

Representative Failure on DataScope. Figure 6 shows a representative failure on DataScope. The task requires sending an SMS to two direct phone numbers and all starred contacts, with the message content derived from a local HTML file. MAI-UI-8B correctly adds the first recipient and produces a coherent intermediate thought. However, it then repeatedly attempts to add the second direct number, cycling between the recipient field, the back button, and the compose screen. By Step 60, the second number is still not confirmed, and the starred contacts have not been processed. This failure arises from missing execution-state awareness. The agent has no persistent record that the first recipient has already been added and confirmed. Because the visible UI state is ambiguous, the agent repeatedly re-evaluates the same situation from interaction history alone and fails to advance to the next subtask. This pattern is especially common in DataScope, where multi-recipient and multi-item workflows require tracking which specific data units have already been processed.

ATMem in a Long-Horizon Trajectory. Figure 4 traces ATMem across a successful 58-step trajectory where the agent transfers three recipes from Simple Gallery Pro to the Broccoli recipe app. During the harvest phase, the agent extracts all recipe fields and stores them as structured entries under `Schema.ItemContent`, each initialized with `status:remaining`. When execution begins, the stored content remains available while the phase field indicates that the agent should start entering recipes into the target app. As each recipe is saved, ATMem updates the corresponding item to `status:finished`. This gives the agent an explicit record of completed and pending items, allowing it to move to the next recipe without re-reading previous screenshots or revisiting completed entries. By the end of the trajectory, all three recipes are marked finished, and the agent terminates correctly. This trajectory illustrates three roles of ATMem in long-horizon execution. During harvest, it stores structured task data that would otherwise need to be recovered from screenshots. During execution, item status provides a checklist for tracking progress. Across item transitions, the memory state helps preserve the correct operation scope, enabling the agent to complete a long multi-item workflow without losing track of pending actions.

4.5. Further Analysis

Training Dynamics under Memory Intervention. Figures 7 (a)–(c) show how ATMem usage evolves during STR-GRPO training and why selective memory invocation emerges. Figure 7 (a) reports outcome rewards under memory-ON and memory-OFF conditions throughout training. The memory-ON curve remains consistently higher than the memory-OFF curve, indicating that ATMem provides a positive contribution when it is retained. Importantly, this gap does not disappear as training progresses. Even after the model learns to invoke memory more selectively, the remaining memory-active cases still achieve higher reward under the memory-ON condition. This suggests that STR-GRPO does not improve performance by simply avoiding memory. Instead, it suppresses low-utility memory updates while preserving cases where structured memory remains beneficial.

Figure 7 (b) compares memory length under STR-GRPO and standard GRPO. Under standard GRPO, memory length remains close to its initial level, suggesting that task-level reward alone provides little pressure to reduce memory content. In contrast, STR-GRPO produces a substantial and sustained reduction in memory length. Read together with Figure 7 (a), this reduction is not accompanied by lower outcome reward. The results suggest that STR-GRPO mainly removes memory updates that provide limited marginal benefit, rather than compressing away useful structured state.

Figure 7 (c) directly measures this effect through the ON-minus-OFF reward gap within each task group. The marginal reward gain from ATMem stays positive across most of the training. It is larger in early training, when memory is used more broadly, and stabilizes at a smaller but sustained level after redundant memory calls are pruned. This pattern is consistent with selective invocation. As low-utility memory updates are removed, the remaining ON-OFF gap reflects the utility of ATMem on tasks that still benefit from explicit state tracking.

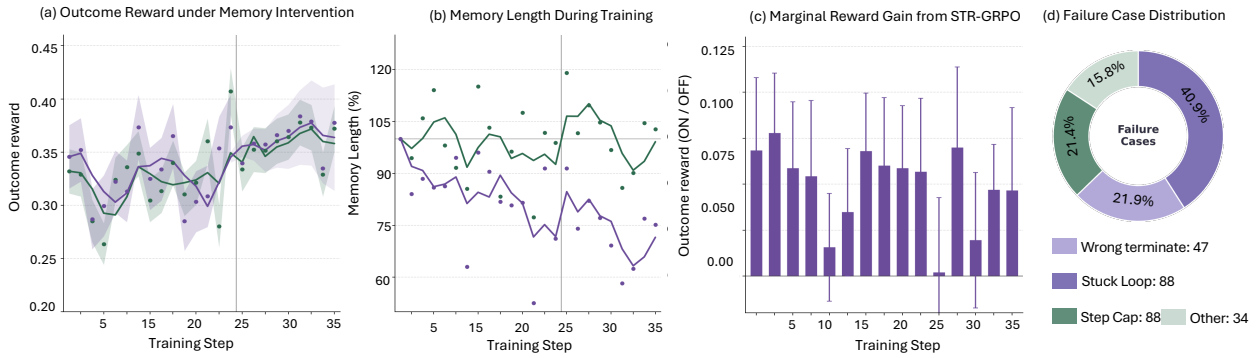

Figure 7: Further analysis of STR-GRPO training dynamics and DataScope failure cases. **(a)** Outcome reward under memory-ON and memory-OFF interventions during training, with shaded regions showing rollout-group variance. Memory-ON consistently achieves higher reward, indicating a sustained benefit from ATMem. **(b)** Normalized memory length relative to training step 1. STR-GRPO substantially reduces memory length, while standard GRPO remains near its initial level. **(c)** Marginal reward gain from ATMem, measured as the outcome-reward difference between memory-ON and memory-OFF cohorts within STR-GRPO. Positive gaps indicate that retained memory invocations provide a measurable benefit. **(d)** Failure type distribution on DataScope for MAI-UI-8B, the strongest end-to-end baseline. Stuck loop is the dominant failure mode at 40.9%, followed by wrong termination (21.9%), step cap (21.4%), and other errors (15.8%).

Failure Mode Analysis on DataScope. Figure 7 (d) breaks down the failure cases of MAI-UI-8B, the strongest end-to-end baseline. Stuck loops are the dominant failure mode at 40.9%, followed by wrong termination (21.9%), step cap (21.4%), and miscellaneous errors, including grounding failures and environment edge cases (15.8%).

The dominance of stuck loops provides a clear diagnosis of why current end-to-end agents struggle on data-scope tasks. Stuck loops occur when the agent cannot determine whether a target data item has already been processed, causing it to repeat the same action or revisit completed sub-tasks. The qualitative example in Figure 6 illustrates this pattern, and the aggregate statistics show that it is a recurring failure mode rather than an isolated case. This is precisely the failure that ATMem is designed to address. By maintaining an explicit execution status for each item, ATMem allows the agent to identify which items remain pending and which have been completed, without relying only on implicit inference from interaction history. Wrong termination and step cap failures reflect different remaining challenges. Wrong termination occurs when the agent declares completion before all required data operations are verified, pointing to limitations in completion checking. Step cap failures occur when the agent remains in a valid but unresolved execution state and fails to converge within the allowed horizon, pointing to limitations in long-horizon planning and recovery. These failures are not fully solved by item-level status tracking alone. They therefore represent the primary remaining challenges for end-to-end agents on data-scope workflows beyond what ATMem directly addresses.

5. Conclusion

We presented ATMem, an active task-driving memory framework for long-horizon mobile GUI agents. Our central argument is that memory in mobile execution should not be treated as a passive archive of past observations. Instead, task-relevant data should be maintained as an evolving execution state that records what has been collected, what remains pending, which constraints apply, and which operations have already been completed. ATMem operationalizes this view by organizing task data into structured units with ownership, role, constraints, and status, allowing the agent to condition its next action on current workflow progress rather than relying only on raw interaction history. To make this memory behavior efficient, we introduced STR-GRPO. By comparing memory-on/off rollouts and incorporating memory cost into online RL, STR-GRPO teaches the agent not only how to construct ATMem, but also when ATMem is actually useful. This turns memory invocation from

an always-active behavior learned through imitation into a utility-driven policy decision. We also introduced DataScope, a long-horizon online benchmark that stresses cross-page and cross-application workflows involving data collection, verification, transfer, update, and reuse. By requiring agents to cover all instruction-relevant targets while filtering same-schema distractors, DataScope exposes failure modes that are difficult to observe from terminal success rate alone. Across AndroidWorld, MobileWorld, and DataScope, ATMem-UI improves long-horizon execution while reducing unnecessary memory usage. Our failure analysis further shows that stuck loops, premature termination, and incomplete operation tracking remain key bottlenecks.

A. Appendix

A.1. Online RL Training Environment and Infrastructure

Long-horizon mobile-agent training requires rollouts to be collected through real multi-turn interactions with the environment. This makes a standard synchronous RL pipeline inefficient, since policy updates must wait for many long and heterogeneous trajectories to finish. The trajectories themselves are also expensive to optimize over: each rollout may concatenate multiple GUI observations, actions, reasoning traces, and ATMem states, resulting in extremely long multimodal contexts that exceed single-GPU memory limits. Following MAI-UI (Zhou et al., 2025), we build an asynchronous on-policy RL framework based on `verl` and HybridFlow (Sheng et al., 2024) to support scalable online training.

Asynchronous rollout collection. We implement a customized agent loop that asynchronously dispatches rollout requests to a pool of inference servers hosting the latest policy model. This design reduces GPU idle time caused by long multi-turn environment interactions and improves rollout throughput. The agent loop also supports asynchronous environment interaction and session management, including backup sessions for failure recovery and replacement. On the inference side, request load balancing and prefill caching are used to accelerate generation under long interaction histories. Although rollouts are collected asynchronously, training remains on-policy: each rollout batch is generated by the current policy version and stored with the corresponding behavior log probabilities for policy optimization.

Hybrid parallelism for long trajectories. To optimize policies over ultra-long trajectories, we adopt Megatron-style hybrid parallelism, including tensor parallelism, pipeline parallelism, and context parallelism. This partitions both model computation and long sequence contexts across multiple GPUs, allowing end-to-end policy updates while keeping per-GPU memory usage manageable. We additionally downsample GUI screenshots to half resolution, which reduces visual-token length and training cost without degrading empirical performance.

Overall, this infrastructure enables stable online RL for long-horizon mobile agents by decoupling rollout collection from policy optimization, improving inference throughput, and supporting training over extremely long multimodal trajectories.

A.2. Training Infrastructure and Hyperparameters

We provide the training infrastructure and main hyperparameters in Table 5. For SFT, we train on verified rollout trajectories using 128 NVIDIA H20 GPUs across 16 nodes. Each node contains 8 GPUs and a 96-core CPU with approximately 900GB of memory. The SFT stage takes about 2 hours. We train the language model while freezing the vision encoder, using bfloat16 precision, FlashAttention, gradient checkpointing, Liger kernel acceleration, and DeepSpeed ZeRO-2 offload.

For online RL, we use the same 128-GPU training configuration and train for about 3 days. The mobile environments are deployed on bare-metal servers. In our main run, we use 128 active environment containers for rollout collection and maintain additional backup containers for failure recovery. Each environment server has 96 CPU cores and 384GB of memory. The rollout system sets the maximum response length to 81,920 tokens. The RL actor is initialized from the SFT checkpoint.

The RL system is built on a `verl`-based multi-turn agent training stack. For generation and policy optimization, we use tensor parallelism with degree 2, context parallelism with degree 2, and pipeline parallelism with degree 1. The rollout engine uses a GPU memory utilization ratio of 0.55, disables replay, and performs checkpointing and evaluation every epoch.

A.3. ATMem-Augmented Agent Execution Prompt

During evaluation, our agent uses the structured execution prompt in Box A.3. It constrains each response to include reasoning, a single executable tool call, and an optional ATMem memory update.

ATMem-Augmented Agent Execution Prompt

*## **Task.*** You are a GUI agent. You are given a task goal, previous action history, and screenshots of the current screen. You need to perform the **NEXT** action to complete the task.

*## **Output Format.*** For each step, output exactly **THREE** blocks in this order:

```
<thinking>
...  return the thinking process
</thinking>
<tool_call>
{"name":  "mobile_use", "arguments":  <args-json-object>}
</tool_call>
<memory>
{ ...valid JSON dict or {} ...  }
</memory>
```

Rules.

- Output exactly **one** tool call per step.

*## **Action Space.*** Choose exactly **one** action per step:

Action	JSON Format
`open_app`	`{"action": "open_app", "app_name": "<Available Apps>"}`
`click`	`{"action": "click", "coordinate": [x, y]}`
`long_press`	`{"action": "long_press", "coordinate": [x, y]}`
`input_text`	`{"action": "input_text", "text": ""}`
`scroll`	`{"action": "scroll", "direction": "up|down|left|right"}`
`drag`	`{"action": "drag", "coordinate": [x, y], "direction": "up|down|left|right"}`
`navigate_back`	`{"action": "navigate_back"}`
`navigate_home`	`{"action": "navigate_home"}`
`wait`	`{"action": "wait"}`
`status`	`{"action": "status", "goal_status": "complete|infeasible"}`
`answer`	`{"action": "answer", "text": ""}`

*## **Hard Constraints.***

- Use `answer` only when the task can be completed without interacting with the device UI.
- Otherwise, use device actions and do not answer directly.
- Output `goal_status="complete"` only after verifying the result on screen.

*## **When to Use Memory.*** Use `<memory>` only when cross-step storage or comparison is needed; otherwise output `{}`.

*## **Memory Schema.*** When used, memory must follow this structure:

```
{
"phase":  "harvest" | "execute",
"context":  {},
"schema":  {
"fields":  []
},
"items":  {
"1":  {
"content":  {},
"status":  "remaining" | "finished" | "skipped",
"skipReason":  ""
}
}
}
```

Rules.

Table 5: Training infrastructure and main hyperparameters.

Configuration	SFT	Online RL
Base model	Qwen3-VL-4B/8B-Instruct	SFT checkpoint
Training GPUs	128 NVIDIA H20	128 NVIDIA H20
Node setup	16 nodes, 8 GPUs/node	16 nodes, 8 GPUs/node
CPU / memory per training node	96 cores / ∼900GB	96 cores / ∼900GB
Training time	∼2 hours	∼3 days
Precision	bfloat16	bfloat16
Max sequence / response length	20,768	81,920
Vision encoder	Frozen	–
Parallelism	ZeRO-2 offload	TP=2, PP=1, CP=2
Batch size	1 per GPU	global 32, mini-batch 32
Rollout group size	–	16
Screenshot resolution	max pixels 12,845,056	1600×720
Environment servers	–	3 bare-metal servers
Environment server config	–	96 cores / 384GB each
Environment containers	–	200 total, 128 active

- Store only structured values in memory; do not store raw text dumps.

Memory Usage Protocol.
- If using memory, start with `phase="harvest"` and switch to `phase="execute"` once enough information is collected.
- Update an item status to `finished` in the same step after a successful operation.

A.4. Data-Centric Benchmark Details

This section provides detailed statistics of our Data-Centric benchmark. Each task family is designed as a data-dependent mobile workflow involving one or more applications, where the agent must identify target data, avoid confusable distractors, and perform the required data operations. Here, confusable distractors refer to non-target entries that share the same schema or data type with target entries and are generated with template-level similarity constraints. To construct controlled difficulty levels, we instantiate the same task families across DC-V1, DC-V2, and DC-V3 while progressively increasing the number of target entries, confusable distractors, and required operations. This design keeps the underlying app workflow largely comparable across subsets, while scaling difficulty through data volume and distractor density.

Table 6 reports task-family-level details. For each subset, **T/N/O** denotes the number of target entries, confusable distractors, and required operations, respectively. The application column lists the apps involved in each workflow using color-coded labels. A "–" indicates that the corresponding task variant is not included in that subset after filtering unreachable or invalid instances.

Table 6: **Task-family details across benchmark subsets.** Each subset cell reports **T/N/O**, where **T**, **N**, and **O** denote the numbers of target entries, same-semantics distractors, and required operations, respectively. Applications are shown as color-coded labels for readability.

Task Family	Apps	#Apps	DC-V1	DC-V2	DC-V3
CalendarFocusTracksRetroPlaylist	Calendar Music	2	0/4/3	0/13/3	0/20/3
CalendarMeetingPrepAndNotify	Calendar	1	5/13/5	20/43/20	36/100/36

Continued on next page

Task Family	Apps	#Apps	DC-V1	DC-V2	DC-V3
CalendarTodayAlarmsInClock	Calendar, Clock	2	2/7/2	3/10/3	4/17/4
CallLogContactTodoMarkorSync	Contacts, Markor, Phone, Tasks	4	2/0/2	3/6/3	3/7/3
CallLogTopContactsFavorite	Contacts, Phone	2	13/1/13	18/14/18	22/29/22
ChromeInviteCodeAndSmsSend	Chrome, Messages	2	3/4/3	4/7/4	9/18/9
ExpensePurgeHousingFoodLogMarkor	Markor, Expense Book	2	0/0/3	0/0/3	0/0/5
FilesClosedProjectContactDelete	Contacts, Files	2	2/4/2	3/12/3	3/35/3
FilesDatedReimburseTxtToExpense	Files, Expense Book	2	3/0/3	3/0/3	4/0/4
FilesPartyMenuRecipesToBroccoli	Broccoli, Files	2	2/0/2	2/0/2	2/0/2
FilesSetlistExportRetroM3u	Files, Music	2	0/3/4	0/6/4	0/9/4
FilesWorkEventsToCalendar	Calendar, Files	2	3/6/3	6/10/6	8/17/8
FilesWorkMergeToMarkorCsv	Files, Markor	2	3/3/3	4/3/4	6/8/6
GalleryImageListToMarkor	Gallery, Markor	2	4/1/4	4/6/4	6/12/6
GalleryVlcArtistPlaylistsFromImages	Gallery, VLC	2	2/3/2	3/12/3	4/21/4
MarkorTaskAssignmentDistribute	Markor	1	2/5/2	4/25/4	5/60/5
RecipeNutFreePurgeLogMarkor	Broccoli, Markor	2	6/3/6	7/3/7	9/3/9
RetroLightMusicPartyMarkorSms	Markor, Messages, Music	3	2/2/2	2/6/2	3/8/3
SmsAgentPhoneToMarkorTxt	Markor, Messages	2	4/10/4	4/7/4	6/30/6
SmsAlbumQueueRetroMarkor	Markor, Messages, Music	3	2/8/2	2/13/2	2/21/2
SmsColleagueTripReimburseToExpense	Messages, Expense Book	2	1/4/1	1/4/1	2/15/10
SmsDavidBugfixToCalendar	Calendar, Messages	2	3/6/3	4/8/4	6/10/6
SmsDavidClientsToContacts	Contacts, Messages	2	3/7/3	3/18/3	4/48/4
SmsDavidTodoToTasks	Messages, Tasks	2	1/3/1	2/10/2	3/25/3
SmsExpenseAuditAndReport	Messages, Expense Book	2	0/2/2	0/6/6	0/15/12
SmsFamilyRecipeToBroccoli	Broccoli, Messages	2	2/4/2	3/9/3	4/13/4
SmsFriendRecommendationsVlcPlaylists	Messages, VLC	2	2/3/2	5/13/5	7/20/7

Continued on next page

Task Family	Apps	#Apps	DC-V1	DC-V2	DC-V3
SmsPotluckRecipeBroccoliCalendar	Broccoli, Calendar, Messages	3	4/6/4	4/6/4	5/9/5
SmsStockDataToMarkorCsv	Markor, Messages	2	2/5/2	4/12/4	7/27/7
SmsWorkPlanToMarkorTodo	Markor, Messages, Tasks	3	1/3/1	3/10/3	5/25/5
SmsWorkoutPlaylistRetroMarkor	Markor, Messages, Music	3	1/3/1	1/9/1	1/20/1
VlcPlaylistsToMarkorMdFiles	Files, Markor, VLC	3	2/8/2	2/11/2	2/17/2

A.5. Example Workflow Template of Our Online Benchmark

To illustrate our data-centric workflow specification, Listing 1 presents a compact example from one of our 32 workflow templates. The example is shortened for readability but preserves the main fields used in benchmark construction. The `task`, `goal`, and `apps` fields define the task family, user-facing instruction, and involved mobile applications. The `custom_params_template` and `initialization_example` specify the concrete data injected into the environment, including target files, app-specific records, and confusable distractors. The `expected_output_example` field defines the required terminal state after successful execution, while `_verification` lists the app-level checks used by the verifier. The `task_extension_suggestions` field further describes how the same workflow can be instantiated with different data values and scaled to harder variants, such as adding more target entries or distractors. Together, these fields bind the task goal, environment initialization, expected outcome, and automatic verification into a single executable task specification.

A.6. Environment Initialization and Verifier Calibration

Following AndroidWorld's (Rawles et al., 2024) app-state-based task construction practice, we implement app-specific initialization and verification functions for each workflow template. For each involved application, the initializer writes the required target data and confusable distractors into the corresponding app storage, such as SQLite-backed records or file-system entries. This ensures that all task-relevant data and distractor data are visible to the agent through normal GUI interaction, rather than being provided only in the instructions.

For each task family, we further implement a task-specific verifier that checks whether the final environment state satisfies the expected outcome. The verifier inspects app-level terminal states, such as created files, modified records, sent messages, calendar entries, contact updates, or untouched distractors, and converts them into rule-level scores. This enables automatic evaluation of both terminal success and partial app-/data-level progress.

To validate verifier reliability, we perform verifier calibration with controlled terminal-state variants. Given the expected result of each task, GPT-5.2 generates multiple perturbed terminal states, including successful completion, missing one or more required data operations, extra operations on non-target data, missing operations in a particular app, and incorrect field values. We then initialize the environment directly with each controlled terminal state, treating it as a simulated post-execution state, and run the corresponding verifier. A verifier is retained only if its score matches the expected score for all controlled variants; otherwise, the task case is returned for manual correction. This calibration process helps ensure that our automatic evaluators correctly distinguish complete success, partial completion, over-operation, and incorrect data manipulation.

A.7. Prompt Template for Benchmark Task Generation

To instantiate diverse task variants from each workflow template, we use an LLM-based task generator with a structured system prompt. The prompt constrains generation to the same logical task class as the source template: the involved applications, cross-app workflow, output types, and structural requirements must remain unchanged, while concrete data values and scenario entities can vary. Difficulty is controlled only through the number of target data entries and confusable distractors, rather than by adding new steps or extra instructions. The prompt also requires each generated variant to include initialization data and expected outputs following the source template schema, so that the resulting task can be initialized in the environment and evaluated automatically by the verifier. A shortened version of the system prompt is shown in Box A.7.

System Prompt for Data-Centric Task Variant Generation

**Task.** Generate feasible GUI automation task variants for an Android long-horizon benchmark.

**Inputs.**
- `requested_num_tasks`: exact number of variants to output.
- `per_variant_difficulty`: ordered list of difficulty labels.
- `full_benchmark_template`: complete benchmark JSON for one task family.
- Optional trajectory screenshots: UI references only.

**Task-Class Constraints.** Generate variants for the **same logical task class** as the source template.
- Keep the same apps, cross-app workflow, and output types.
- Scenario wording and concrete values may change, but apps, steps, and structural requirements must not be added or removed.
- `task_goal` and `cn_task_goal` should remain concise, specific, and agent-actionable.

**Difficulty Model.** Difficulty is increased only through **target data volume** and **confusable distractor density**, not by adding new steps or extra instructions.

Difficulty	Operational / Target Data	Noise / Distractor Data
`easy`	1–2 target sub-groups.	2–5 noise items, clearly distinct from targets.
`medium`	2–3 target sub-groups; roughly 1.5× easy counts.	5–8 noise items with structural similarity.
`hard`	3–5 target sub-groups; roughly 2× easy counts.	8–12 highly similar noise items, including near-duplicate names, formats, or partial-match fields.
`very_hard`	5–8 target sub-groups; roughly 3× easy counts.	12–20 highly confusable noise items, including trap entries with one missing or close-but-wrong field.

**Reference and Ground Truth.** When an `lh_v1_reference` is provided, use it as the authoritative reference for:
1. `task_goal` and `cn_task_goal` phrasing;
2. `expected_output` structure, including top-level keys, nesting, and field names;
3. `init_data.custom_params` schema.

Only concrete values, counts, and scenario entities may change.

**Expected Output Rules.**
- Fill `expected_output` to exactly match what a correct agent should produce for the current variant.
- Use the source template's `expected_output_example` as the structural reference.
- Do not emit `rule_validation`; it is derived automatically from `expected_output`.
- Do not emit keys starting with `_` inside `expected_output`.

**Initialization Data.**
- `init_data` must not be null.
- Follow the structural layout of `full_benchmark_template.initialization_example`.
- Ensure unique names, phone numbers, and IDs for all contacts or records.
- Strip all `_`-prefixed keys from emitted initialization data.

**Output Format.** Return a single valid JSON object only. Do not include markdown fences, comments, or extra text.

```
{
"task_type": "<exact task type from source template>",
"variants": [
{
"variant_id": 1,
"difficulty": "easy|medium|hard|very_hard",
"task_goal": "<same task class; values may change>",
"cn_task_goal": "<Chinese task goal>",
"init_data": {
"task_type": "<same as task_type>",
"custom_params": { "<seeded data fields from template>" }
},
"expected_output": { "<ground truth with reference structure>" },
"rationale": "<=120 words describing target/noise counts>"
}
]
}
```

```
{
  "task": "CalendarMeetingPrepAndNotify",
  "goal": "Check next week's meetings in Simple Calendar Pro. For each target meeting, find related pending tasks in Tasks
     whose notes mention the meeting title, create a Markor agenda file named {meeting_title}_agenda.md, look up attendee
     phone numbers in Contacts, and send the agenda via Simple SMS Messenger.",
  "apps": [
    "Simple Calendar Pro",
    "Tasks",
    "Contacts",
    "Markor",
    "Simple SMS Messenger"
  ],
  "task_type": "long-horizon",

  "_why_hard": [
    "Five apps are chained: Calendar -> Tasks -> Markor -> Contacts -> SMS.",
    "Target meetings, tasks, and contacts are mixed with confusable distractors.",
    "The SMS body must match the corresponding meeting agenda."
  ],

  "task_extension_suggestions": {
    "1": "Increase the number of target meetings; each meeting requires an independent agenda file and SMS messages.",
    "2": "Increase noise_events, noise_tasks, and noise_contacts.",
    "3": "Add attendees without matching contacts to test skipping or fault tolerance.",
    "4": "Require strict agenda filename or section formatting."
  },

  "difficulty_control": {
    "easy": {
      "target_meetings": 1,
      "attendees_per_meeting": 2,
      "related_tasks_per_meeting": 2,
      "noise_events": 3,
      "noise_tasks": 5,
      "noise_contacts": 5
    },
    "medium": {
      "target_meetings": "2-3",
      "attendees_per_meeting": "2-4",
      "related_tasks_per_meeting": "2-5",
      "noise_events": 8,
      "noise_tasks": 15,
      "noise_contacts": 20
    },
    "hard": {
      "target_meetings": "3-4",
      "attendees_per_meeting": "3-6",
      "related_tasks_per_meeting": "3-8",
      "noise_events": 15,
      "noise_tasks": 35,
      "noise_contacts": 50
    }
  },

  "custom_params_template": {
    "calendar_events": {
      "target_events": [
        {
          "title": "Vendor Contract Sync",
          "date": "2023-10-24",
          "time": "10:00",
          "duration_mins": 60,
          "location": "Meeting Room 2B",
          "description": "Align on contract renewal timeline and action items.",
          "attendees": ["Nina Patel", "Omar Reyes"]
        }
      ],
      "noise_events": [
        {
          "title": "Team Lunch",
          "date": "2023-10-25",
          "time": "12:00",
```

```
        "duration_mins": 60,
        "description": "Lunch at the cafe.",
        "attendees": []
      }
    ]
  },

  "tasks_app_data": {
    "target_tasks": [
      {
        "title": "Draft agenda for vendor sync",
        "completed": 0,
        "deleted": 0,
        "notes": "For Vendor Contract Sync: propose topics and timing."
      },
      {
        "title": "Collect renewal milestones",
        "completed": 0,
        "deleted": 0,
        "notes": "Needed for Vendor Contract Sync."
      }
    ],
    "noise_tasks": [
      {
        "title": "Buy dish soap",
        "completed": 0,
        "deleted": 0,
        "notes": null
      }
    ]
  },

  "contacts_data": {
    "target_contacts": [
      {
        "name": "Nina Patel",
        "number": "+13661100111"
      },
      {
        "name": "Omar Reyes",
        "number": "+13661100112"
      }
    ],
    "noise_contacts": [
      {
        "name": "Mia Chen",
        "number": "+13661100202"
      }
    ]
  },

  "output_config": {
    "markor_file_template": "{meeting_title}_agenda.md",
    "markor_content_format": "# {meeting_title}\n- Date: {date}\n- Time: {time}\n- Location: {location}\n- Attendees: {
   attendees}\n\n## Related Tasks\n{task_list}",
    "sms_format": "Agenda for {meeting_title} ({date} {time}): {task_list}. Location: {location}."
  }
},

"initialization_example": {
  "task_type": "CalendarMeetingPrepAndNotify",
  "task_idx": 0,
  "custom_params": {
    "calendar_events": "instantiated from custom_params_template.calendar_events",
    "tasks_app_data": "instantiated from custom_params_template.tasks_app_data",
    "contacts_data": "instantiated from custom_params_template.contacts_data",
    "output_config": "instantiated from custom_params_template.output_config"
  }
},

"expected_output_example": {
  "markor_files": [
    {
```

```
        "file_name": "Vendor_Contract_Sync_agenda.md",
        "content": "# Vendor Contract Sync\n- Date: 2023-10-24\n- Time: 10:00\n- Location: Meeting Room 2B\n- Attendees: Nina
       Patel, Omar Reyes\n\n## Related Tasks\n- Draft agenda for vendor sync\n- Collect renewal milestones\n"
       }
     ],
     "sms_sent": [
       {
         "to": "Nina Patel (+13661100111)",
         "for_meeting": "Vendor Contract Sync",
         "body": "Agenda for Vendor Contract Sync: Draft agenda for vendor sync; Collect renewal milestones. Location: Meeting
       Room 2B."
       },
       {
         "to": "Omar Reyes (+13661100112)",
         "for_meeting": "Vendor Contract Sync",
         "body": "Agenda for Vendor Contract Sync: Draft agenda for vendor sync; Collect renewal milestones. Location: Meeting
       Room 2B."
       }
     ]
   }
}
```


Listing 1: Compact example of a data-centric workflow template. Arrays are shortened for readability while preserving the main template structure.

References

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. 10

Yuxiang Chai, Hanhao Li, Jiayu Zhang, Liang Liu, Guangyi Liu, Guozhi Wang, Shuai Ren, Siyuan Huang, and Hongsheng Li. A3: Android agent arena for mobile gui agents. *arXiv preprint arXiv:2501.01149*, 2025. 2

Jingxuan Chen, Derek Yuen, Bin Xie, Yuhao Yang, Gongwei Chen, Zhihao Wu, Li Yixing, Xurui Zhou, Weiwen Liu, Shuai Wang, et al. Spa-bench: A comprehensive benchmark for smartphone agent evaluation. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024. 2

Liangyu Chen, Hanzhang Zhou, Chenglin Cai, Jianan Zhang, Panrong Tong, Quyu Kong, Xu Zhang, Chen Liu, Yuqi Liu, Wenxuan Wang, et al. Ui-ins: Enhancing gui grounding with multi-perspective instruction-as-reasoning. *arXiv preprint arXiv:2510.20286*, 2025. 6, 10, 11

Qijia Chen, Andrea Bellucci, Zhida Sun, and Giulio Jacucci. Skilldroid: Compile once, reuse forever. *arXiv preprint arXiv:2604.14872*, 2026. 4

Weihua Cheng, Ersheng Ni, Wenlong Wang, Yifei Sun, Junming Liu, Wangyu Shen, Yirong Chen, Botian Shi, and Ding Wang. Mga: Memory-driven gui agent for observation-centric interaction. *arXiv preprint arXiv:2510.24168*, 2025. 4

DeepMind. Developing a computer use model. Google Blog, Oct 2025. URL https://blog.google/technology/google-deepmind/gemini-computer-use-model/. Accessed: October 22, 2025. 10

Shihan Deng, Weikai Xu, Hongda Sun, Wei Liu, Tao Tan, Liujianfeng Liujianfeng, Ang Li, Jian Luan, Bin Wang, Rui Yan, et al. Mobile-bench: An evaluation benchmark for llm-based mobile agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8813–8831, 2024. 2

Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025. 3, 10, 11

Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, et al. Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments. *arXiv preprint arXiv:2512.19432*, 2025. 2, 4

Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 1119–1133, 2024. 4

Runze Li, Yuwen Zhai, Bo Xu, LiWu Xu, Nian Shi, Wei Zhang, Ran Lin, and Liang Wang. Echotrail-gui: Building actionable memory for gui agents via critic-guided self-exploration. *arXiv preprint arXiv:2512.19396*, 2025. 4

Yanda Li, Chi Zhang, Wenjia Jiang, Wanqi Yang, Bin Fu, Pei Cheng, Xin Chen, Ling Chen, and Yunchao Wei. Appagent v2: Advanced agent for flexible mobile interactions. *arXiv preprint arXiv:2408.11824*, 2024. 4

Guangyi Liu, Pengxiang Zhao, Yaozhen Liang, Qinyi Luo, Shunye Tang, Yuxiang Chai, Weifeng Lin, Han Xiao, WenHao Wang, Siheng Chen, et al. Memgui-bench: Benchmarking memory of mobile gui agents in dynamic environments. *arXiv preprint arXiv:2602.06075*, 2026. 4

Team OpenAI. Introducing openai o3 and o4-mini. *https://openai. com/index/introducing-o3-and-o4-mini/*, 2025. 6, 10, 11

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023. 2

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. 3, 10

Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023. 2

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024. 2, 4, 21

Daniel L Schacter. Constructive memory: past and future. *Dialogues in clinical neuroscience*, 14(1):7–18, 2012. 2

Daniel L Schacter and Donna Rose Addis. The cognitive neuroscience of constructive memory: Remembering the past and imagining the future. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 362(1481):773, 2007. 2

Bytedance Seed. Seed1.8 model card: Towards generalized real-world agency. *arXiv preprint*, December 2025a. Technical Report. 10

ByteDance Seed. Ui-tars-1.5. https://seed-tars.com/1.5, 2025b. 10, 11

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024. 17

Yibo Shi, Jungang Li, Linghao Zhang, Zihao Dongfang, Biao Wu, Sicheng Tao, Yibo Yan, Chenxi Qin, Weiting Liu, Zhixin Lin, et al. Androtmem: From interaction trajectories to anchored memory in long-horizon gui agents. *arXiv preprint arXiv:2603.18429*, 2026. 4

Yucheng Shi, Wenhao Yu, Zaitang Li, Yonglin Wang, Hongming Zhang, Ninghao Liu, Haitao Mi, and Dong Yu. Mobilegui-rl: Advancing mobile gui agent through reinforcement learning in online environment. *arXiv preprint arXiv:2507.05720*, 2025. 4

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023. 2

Theodore Sumers, Shunyu Yao, Karthik R Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023. 2

Jiazheng Sun, Te Yang, Jiayang Niu, Mingxuan Li, Yongyong Lu, Ruimeng Yang, and Xin Peng. Fairy: Interactive mobile assistant to real-world tasks via lmm-based multi-agent. *arXiv e-prints*, pages arXiv–2509, 2025. 4

Libo Sun, Jiwen Zhang, Siyuan Wang, and Zhongyu Wei. Magnet: Towards adaptive gui agents with memory-driven knowledge evolution. *arXiv preprint arXiv:2601.19199*, 2026. 4

GELab Team. Gelab-zero: An advanced mobile agent inference system, 2025. URL https://github.com/stepfun-ai/gelab-zero. 3, 10, 11

Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, et al. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*, 2026. 10, 11

Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhui Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. Agentprog: Empowering long-horizon gui agents with program-guided context management. *arXiv preprint arXiv:2512.10371*, 2025. 4

Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a. 3, 10

Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration. *Advances in Neural Information Processing Systems*, 37:2686–2710, 2024a. 2

Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024b. 2

Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. Mobile-agent-e: Self-evolving mobile assistant for complex tasks. *arXiv preprint arXiv:2501.11733*, 2025b. 2, 4, 14

Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th annual international conference on Mobile computing and networking*, pages 543–557, 2024. 3, 4

Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. Ui-mem: Self-evolving experience memory for online reinforcement learning in mobile gui agents. *arXiv preprint arXiv:2602.05832*, 2026. 2, 4

Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*, 2026. 3, 4, 10, 11

Yifan Xu, Xiao Liu, Xinghan Liu, Jiaqi Fu, Hanchen Zhang, Bohao Jing, Shudan Zhang, Yuting Wang, Wenyi Zhao, and Yuxiao Dong. Mobilerl: Online agentic reinforcement learning for mobile gui agents. *arXiv preprint arXiv:2509.18119*, 2025a. 4

Yifan Xu, Xiao Liu, Xueqiao Sun, Siyi Cheng, Hao Yu, Hanyu Lai, Shudan Zhang, Dan Zhang, Jie Tang, and Yuxiao Dong. Androidlab: Training and systematic benchmarking of android autonomous agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2144–2166, 2025b. 4

Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, Shiliang Yang, Zhirui Wang, Brian Li, Kang An, Chenyang Li, Lei Lei, Mengmeng Duan, Danxun Liang, Guodong Liu, Hang Cheng, Hao Wu, Jie Dong, Junhao Huang, Mei Chen, Renjie Yu, Shunshan Li, Xu Zhou, Yiting Dai, Yineng Deng, Yingdan Liang, Zelin Chen, Wen Sun, Chengxu Yan, Chunqin Xu, Dong Li, Fengqiong Xiao, Guanghao Fan, Guopeng Li, Guozhen Peng, Hongbing Li, Hang Li, Hongming Chen, Jingjing Xie, Jianyong Li, Jingyang Zhang, Jiaju Ren, Jiayu Yuan, Jianpeng Yin, Kai Cao, Liang Zhao, Liguo Tan, Liying Shi, Mengqiang Ren, Min Xu, Manjiao Liu, Mao Luo, Mingxin Wan, Na Wang, Nan Wu, Ning Wang, Peiyao Ma, Qingzhou Zhang, Qiao Wang, Qinlin Zeng, Qiong Gao, Qiongyao Li, Shangwu Zhong, Shuli Gao, Shaofan Liu, Shisi Gao, Shuang Luo, Xingbin Liu, Xiaojia Liu, Xiaojie Hou, Xin Liu, Xuanti Feng, Xuedan Cai, Xuan Wen, Xianwei Zhu, Xin Liang, Xin Liu, Xin Zhou, Yingxiu Zhao, Yukang Shi, Yunfang Xu, Yuqing Zeng, Yixun Zhang, Zejia Weng, Zhonghao Yan, Zhiguo Huang, Zhuoyu Wang, Zheng Ge, Jing Li, Yibo Zhu, Binxing Jiao, Xiangyu Zhang, and Daxin Jiang. Step-gui technical report, 2025. URL https://arxiv.org/abs/2512.15431. 2, 10

Leyang Yang, Ziwei Wang, Xiaoxuan Tang, Sheng Zhou, Dajun Chen, Wei Jiang, and Yong Li. Probench: Benchmarking gui agents with accurate process information. *arXiv preprint arXiv:2511.09157*, 2025. 2

Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025. 2, 4, 10, 11

Mingxian Yu, Siqi Luo, and Xu Chen. Graphpilot: Gui task automation with one-step llm reasoning powered by knowledge graph. *arXiv preprint arXiv:2601.17418*, 2026. 4

Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–20, 2025. 2, 4

Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025. 3, 4, 10, 11, 17

Sibo Zhu, Wenyi Wu, Kun Zhou, Stephen Wang, and Biwei Huang. Hybrid self-evolving structured memory for gui agents. *arXiv preprint arXiv:2603.10291*, 2026. 4

Zichen Zhu, Hao Tang, Yansi Li, Dingye Liu, Hongshen Xu, Kunyao Lan, Danyang Zhang, Yixuan Jiang, Hao Zhou, Chenrun Wang, et al. Moba: multifaceted memory-enhanced adaptive planning for efficient mobile task automation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*, pages 535–549, 2025. 4